

```

• mappings.push({window: chromeWindow, originalFunction:
originalFunction});
•
• XULBrowserWindow.hideChromeForLocation =
function(aLocation) {
• return originalFunction.call(XULBrowserWindow, aLoca-
tion) ||
• chromeWindow.gBrowser.selectedTab.pinned;
• }
•
• chromeWindow.document.addEventListener("TabPinned",
triggerOnLocationChange, false);
• chromeWindow.document.addEventListener("TabUnpinned",
triggerOnLocationChange, false);
• }
•
• function stop(chromeWindow) {
• for (let i in mappings) {
• let pair = mappings[i];
• if (chromeWindow === pair.window) {
• chromeWindow.XULBrowserWindow.hideChromeForLocation
= pair.originalFunction;
• chromeWindow.document.removeEventListener("TabPi
nned", triggerOnLocationChange, false);
• chromeWindow.document.removeEventListener("TabUnpin
ned", triggerOnLocationChange, false);
• mappings.splice(i,1);
• break;
• }
• }
• }
• }
•
• </code>

```

What, what? Did we just see XUL there somewhere? Well, yes. As we'll examine in a minute, XUL and Jetpack aren't so much rival models as they're complementary. Let's see how. First, we notice that 'wu' is taken as a holder for 'window-util' API from Jetpack. Then, a couple of functions are defined, which would expect windows in their arguments that start and stop the effects of the add-on. Their purpose will become clearer when we look at the rest of the code.